

REDISEÑO DE ARQUITECTURA TECNOLÓGICA

SOONER — PLATAFORMA SAAS DE GESTIÓN DEPORTIVA AUTOMATIZADA

1. Resumen Ejecutivo y Objetivos

El presente documento define y justifica el cambio estratégico en la infraestructura tecnológica de **Sooner**, la solución SaaS destinada a la automatización de fichas, inscripciones y gestión integral de clubes deportivos.

Actualmente, la plataforma está vinculada de manera monolítica a los servicios de Firebase (Firestore, Storage, Cloud Functions). Si bien este enfoque aceleró las fases iniciales de prototipado, las restricciones del modelo NoSQL y el riesgo de costes variables por volumen de lectura/escritura penalizan la escalabilidad de un software con alta interconexión de datos.

Objetivos clave del rediseño:

- **Optimización de costes a \$0 iniciales:** Implementar una arquitectura robusta aprovechando los niveles gratuitos más competitivos del mercado actual, blindando el negocio ante picos de facturación inesperados.
- **Adopción de un Modelo Relacional (SQL):** Estructurar de forma nativa las entidades del club (Clubs, Equipos, Jugadores, Fichas, Pagos) para permitir consultas complejas e íntegras sin duplicidad de datos.
- **Desacoplamiento e independencia:** Separar la lógica y almacenamiento de datos (**Supabase**) de la distribución y alojamiento de la interfaz web (**Vercel**).

2. Comparativa de Ecosistemas: Firebase vs. Supabase + Vercel

A continuación se analiza el impacto del cambio técnico en los diferentes pilares de la aplicación:

Servicio	Arquitectura Actual (Firebase)	Nueva Arquitectura (Supabase + Vercel)	Impacto y Ventaja Crítica
Base de Datos	Firestore (NoSQL): Almacenamiento en documentos JSON. Sin relaciones nativas. Coste basado en cantidad de operaciones de lectura/escritura.	PostgreSQL (SQL): Relacional nativo. Operaciones complejas mediante JOINS. Coste basado en almacenamiento fijo.	Eficiencia radical en consultas complejas (ej: listados de deudores por equipo). Costes predecibles.
Autenticación	Firebase Auth: Correcto, integrado. Gratuito para flujos básicos de email/contraseña.	Supabase Auth: Basado en JWT, integrado directamente con las políticas de seguridad de PostgreSQL (RLS).	Permite proteger filas de la base de datos de manera automática según el rol del usuario conectado.
Almacenamiento	Firebase Storage: Almacenamiento de archivos (fotos de fichas, documentos). Capa gratuita de 5 GB.	Supabase Storage: Almacenamiento de objetos. Capa gratuita de 1 GB (ampliable o combinable).	Suficiente para el arranque. Mantiene la misma lógica de buckets y URLs firmadas de forma segura.
Hosting Frontend	Firebase Hosting: Capa gratuita limitada a 10 GB de transferencia mensual.	Vercel Hosting: Capa gratuita de 100 GB de transferencia mensual. CI/CD automático con GitHub.	Multiplica por 10 la capacidad gratuita de tráfico web. Automatiza los despliegues al hacer git push.

3. Diseño del Modelo de Datos Relacional (PostgreSQL)

A diferencia de la naturaleza permisiva de Firestore, PostgreSQL exige la definición de un esquema de datos rígido pero altamente eficiente. Esto garantiza que no existan inconsistencias (como una ficha huérfana de jugador o un pago asignado a un equipo inexistente).

El esquema inicial sugerido para la base de datos de **Sooner** se estructura de la siguiente manera:

- **clubs:** ID único, nombre, logotipo, datos fiscales, ID del administrador principal.
- **teams:** ID único, club_id (Clave foránea hacia clubs), nombre, categoría, temporada.
- **players:** ID único, nombre, apellidos, fecha de nacimiento, DNI/Pasaporte, datos del tutor (si es menor).
- **team_players:** Tabla intermedia para gestionar qué jugadores pertenecen a qué equipos. Incluye player_id y team_id.

- **registrations (Fichas):** ID único, `player_id`, estado de la ficha (Pendiente, Enviada, Aprobada, Rechazada), documento PDF/Imagen firmado, fecha de alta.
- **payments:** ID único, `player_id`, `club_id`, concepto (Cuota mensual, Matrícula, Equipación), importe, estado (Pendiente, Pagado, Devuelto), fecha de vencimiento, ID de pasarela (Stripe/Sooner Pay).

Ventaja Operativa Inmediata:

Para obtener el total de dinero pendiente de cobro en el "Equipo Juvenil A", SQL ejecuta una sola operación interna sumando la columna `importe` filtrando por el ID del equipo. En Firebase, habrías tenido que descargar todos los documentos de los jugadores del equipo y sumar los costes en el dispositivo del usuario, consumiendo cientos de lecturas innecesarias.

4. Seguridad Avanzada mediante RLS (Row Level Security)

Uno de los mayores retos en un SaaS multi-inquilino (donde conviven múltiples clubes independientes) es asegurar de forma infalible que el Club A jamás pueda leer ni modificar los datos del Club B.

Supabase resuelve esto a nivel de motor de base de datos usando **Row Level Security (RLS)**. En lugar de escribir complejas reglas de validación en el código de la app o en archivos externos, se declaran políticas directamente sobre las tablas de PostgreSQL.

Por ejemplo, la política para leer datos de la tabla de jugadores asegura de forma nativa el aislamiento:

```
CREATE POLICY "Los administradores solo ven jugadores de su propio club"
ON public.players
FOR SELECT
USING ( club_id = (SELECT club_id FROM public.users_profiles WHERE auth_id = auth.uid()) );
```

5. Plan de Viabilidad Económica (Estrategia \$0 Inicial)

El despliegue conjunto de Supabase y Vercel permite mitigar por completo los costes fijos y variables durante la fase de lanzamiento (MVP):

1. **Infraestructura sin costes fijos:** Tanto el hosting de la aplicación en Vercel como las bases de datos de Supabase operan bajo planes gratuitos sin necesidad de introducir tarjeta de crédito obligatoriamente para el desarrollo.
2. **Protección contra picos de uso:** Si el software experimenta un incremento de tráfico repentino (por ejemplo, durante la semana de inscripciones de principio de temporada), Vercel absorberá los accesos gracias a sus 100 GB de ancho de banda gratuito. Si se supera el límite de espacio en la base de datos de Supabase (500 MB de texto), la plataforma avisa con antelación para realizar un upgrade controlado a un plan de precio fijo (\$25/mes), eliminando las sorpresas de cobros por uso indexado de Firebase.

6. Gestión de Credenciales y Variables de Entorno (.env)

Para que el frontend alojado en Vercel pueda comunicarse de manera segura con el backend de Supabase sin exponer claves maestras o romper la seguridad de la aplicación, es fundamental delegar esta configuración en las **Variables de Entorno**. El flujo de conexión requiere dos claves principales que se obtienen desde el panel de control de Supabase (*Project Settings -> API*):

- **SUPABASE_URL**: La dirección IP/URL única de tu instancia de base de datos. Es pública y necesaria para que el cliente JavaScript sepa a dónde enviar las peticiones.
- **SUPABASE_ANON_KEY**: La clave pública anónima. Es totalmente segura de incluir en el código del navegador, ya que respeta de forma estricta las reglas RLS configuradas en la base de datos.

Inyección segura en Vercel: Durante el proceso de vinculación de tu repositorio en Vercel, la plataforma ofrece un apartado específico llamado *Environment Variables*. Al pegar estas claves allí, Vercel las cifrará y las inyectará automáticamente en el código compilado en producción, eliminando por completo la necesidad de subir archivos `.env` locales a GitHub (lo cual comprometería la seguridad del código).

Alerta Crítica de Seguridad:

Bajo ninguna circunstancia debes incluir o exponer la clave llamada `service_role` (secret key) en Vercel o en el frontend de tu app. Esta clave tiene permisos de superadministrador, salta todas las políticas RLS y da acceso total e ilimitado de lectura y borrado a las tablas.

7. Próximos Pasos para la Ejecución

Para llevar a cabo la transformación del código de la app de manera segura, el proceso debe seguir el siguiente orden secuencial:

- **Paso 1:** Diseñar y ejecutar el script SQL de creación de tablas en la consola de Supabase.
- **Paso 2:** Extraer las credenciales de API de Supabase y configurar el entorno local.
- **Paso 3:** Reemplazar la inicialización del SDK de Firebase por el cliente de Supabase (`@supabase/supabase-js`).
- **Paso 4:** Sustituir el flujo de autenticación (los métodos de login/registro).
- **Paso 5:** Migrar de forma progresiva las consultas de colecciones de Firestore a consultas selectores de Supabase.
- **Paso 6:** Conectar el repositorio de GitHub a Vercel, inyectar las variables de entorno configuradas y lanzar a producción.